

URL DOWNLOADER

Postman/Newman Feature-by-Feature Live API Testing Report

Actual code inspection + live endpoint execution + automated test evidence

Audit item	Value
Execution date	06 October 2026, Asia/Calcutta
Backend tested	FastAPI at http://127.0.0.1:8001
Frontend tested	Next.js at http://127.0.0.1:3000
Repository revision	13a0ce4
Test approach	Postman collection via Newman, live SSE/file workflow, source trace, pytest

This report distinguishes observed results from code-derived expectations. No unexecuted test is marked as passed.

Executive Summary

The primary implementation is a Next.js 16 frontend connected to a FastAPI 0.115.6 backend. The backend uses in-memory download metadata, background asyncio tasks, streamed HTTP downloads, yt-dlp for media inspection/downloads, disk storage for completed files, and Server-Sent Events (SSE) for live progress.

The main download workflow was executed end to end through a Postman collection using Newman against a real public PDF. The API accepted the job, transitioned through live states, downloaded 13,264 bytes, served the completed file as an attachment, rejected cancellation after completion, deleted the file and record, and returned 404 after deletion.

Outcome at a glance

Area	Observed outcome	Assessment
API discovery	9 operational backend routes plus root/docs surfaces discovered from code and OpenAPI	Complete
Real direct download	202 -> validating -> downloading -> completed; 13,264 bytes retrieved	Pass
Media inspection	Public YouTube URL returned title, duration, thumbnail and 4 formats	Pass
Live progress	SSE returned text/event-stream and three state events	Pass
Validation	Schema boundaries return 422; invalid status returns 400	Pass with consistency issues
Security	Direct downloads block loopback asynchronously, but media-info reads loopback URLs	High-severity failure
Cancellation	Request returned 200, then the job became failed rather than cancelled	Defect
Postman/Newman	36 requests, 48 assertions, 46 passed, 2 release-gate failures	Completed with 2 confirmed defects
Python tests	52 passed in 0.39 seconds; tests are mostly utility/unit scope	Pass with coverage gap
Frontend build	Production BUILD_ID generated	Pass
Lint	ESLint 9 could not find eslint.config.*	Tooling defect

Highest-priority actions

- Apply the same SSRF validation to /api/downloads/media-info before yt-dlp is called.
- Make cancellation authoritative and return the post-cancellation state, not the pre-cancellation record.
- Validate URL format synchronously before returning 202, or update the API contract to describe asynchronous validation.
- Add authentication and ownership authorization before exposing the service outside a trusted local environment.
- Add endpoint-level automated tests and repair the ESLint 9 configuration.

Postman/Newman Live Run

A dedicated Postman Collection v2.1 was executed with Newman 6.2.2 against <http://127.0.0.1:8001>. The collection is organized into six feature folders and uses collection variables to chain real download IDs across create, status, SSE, file, cancel, and delete requests.

Metric	Observed result
Collection	URL Downloader - Feature-by-Feature Live Audit
Runner	Newman 6.2.2
Requests	36 executed, 0 request transport failures
Test scripts	36 executed
Pre-request scripts	3 executed
Assertions	48 executed: 46 passed, 2 failed
Run duration	9.2 seconds
Response time	126 ms average; 2 ms minimum; 4.1 s maximum
Data received	Approximately 34.91 kB, excluding the separately verified file size presentation

Release-gate failures

Assertion	Expected	Actual	Meaning
SECURITY: loopback media URL must be rejected	HTTP 400 or 403	HTTP 200	Confirmed SSRF bypass in media-info
LIFECYCLE: accepted cancellation ends as cancelled	status=cancelled	status=failed	Confirmed cancellation race

Feature folders executed

- 01 Service and Documentation - root, health, OpenAPI, Swagger, ReDoc.
- 02 Media Inspection - public media, malformed URL, loopback security gate.
- 03 Direct Download Lifecycle - create, poll, SSE, file, terminal cancel, delete, confirm deletion.
- 04 Listing and Validation - list, filters, pagination bounds, request schema failures.
- 05 Missing Resource Behavior - status, events, file, cancel, and delete 404 cases.
- 06 Cancellation and Active-State Guards - cancel race, incomplete file guard, active delete guard, cleanup.

The two failed assertions are intentional: they preserve the expected security and lifecycle contracts and turn known defects into visible regression gates.

Scope, Architecture, and Evidence

System under test

Layer	Implementation	Evidence
Frontend	Next.js 16.2.11, React 19, TypeScript, Tailwind UI	src/app/page.tsx, components, hooks
API client	Relative /api calls with XTransformPort=8001 routing	src/lib/api.ts and src/lib/config.ts
Backend	FastAPI, Pydantic, httpx, uvicorn	mini-services/url-downloader-backend/app
Media	yt-dlp plus bundled FFmpeg	app/services/media_service.py
Persistence	In-memory repository; files on local disk	download_repository.py and file_service.py
Progress	SSE with 15-second heartbeats; frontend polling fallback	downloads.py and useDownloadEvents.ts
Security	Scheme/DNS/IP SSRF checks and file/MIME allowlists	core/security.py and url_validator.py
Prototype	Separate simple-downloader/backend/main.py	Not used by the current frontend

Execution environment

- Repository revision: 13a0ce4.
- Python 3.14.6; FastAPI 0.115.6; Pydantic 2.13.4; httpx 0.28.1.
- Node.js 26.5.0; npm 11.17.0.
- Backend live at 127.0.0.1:8001 and frontend live at 127.0.0.1:3000.
- Public test inputs: W3C dummy PDF and the media URL already present in the Postman collection.

Evidence rules

Observed means an HTTP request or executable test was run during this audit. Code-verified means the path was traced in source but not forced live. Recommended means a future test or change. This prevents assumptions from being presented as execution results.

Complete Endpoint Inventory

Method	Path	Feature	Live result
GET	/	API information	200 JSON
GET	/api/health	Health	200 healthy
POST	/api/downloads/media-info	Media discovery	200 public media; 400 malformed URL; security failure on loopback
POST	/api/downloads	Create download	202 accepted; 422 schema errors
GET	/api/downloads	History/filter/pagination	200; 400 invalid status; 422 invalid bounds
GET	/api/downloads/{id}	Status detail	200 existing; 404 missing/deleted
GET	/api/downloads/{id}/events	Live SSE progress	200 event stream; 404 missing
GET	/api/downloads/{id}/file	File retrieval	200 file; 409 incomplete; 404 missing
POST	/api/downloads/{id}/cancel	Cancellation	200 active; 409 terminal; 404 missing
DELETE	/api/downloads/{id}	Record/file deletion	204 terminal; 409 active; 404 missing
GET	/openapi.json	Machine-readable contract	200 OpenAPI 3.1
GET	/docs	Swagger UI	200 HTML
GET	/redoc	ReDoc UI	200 HTML
GET	frontend /api	Next.js sample route	200 {message: Hello, world!}

Authentication endpoints do not exist. No API route applies identity, role, token, session, or ownership checks.

1. Service Discovery and Health

GET / and GET /api/health

Feature purpose

The root advertises the service name, version, docs, health path, and download path. Health provides a lightweight readiness signal.

How the frontend uses it

The page calls health before loading download history and displays a backend-unavailable warning when it fails.

Request to send

```
GET http://127.0.0.1:8001/  
GET http://127.0.0.1:8001/api/health
```

Observed positive output

```
HTTP 200  
{"status":"healthy"}  
Root: name="URL Application Downloader", version="1.0.0"
```

Observed negative or boundary output

No negative health behavior is implemented; health does not verify disk writability, network access, repository health, or yt-dlp readiness.

Result

PASS - Both discovery and health responded successfully.

Manual testing notes

- Use health for process availability only, not deep dependency readiness.
- Add a detailed readiness endpoint if deployment automation needs dependency checks.

2. API Contract and Interactive Documentation

GET /openapi.json, /docs, /redoc

Feature purpose

FastAPI generates an OpenAPI 3.1 contract and two interactive documentation surfaces.

How the frontend uses it

The user interface does not consume these routes; they support developers, QA, Postman import, and contract review.

Request to send

```
GET /openapi.json
GET /docs
GET /redoc
```

Observed positive output

All three returned HTTP 200. OpenAPI listed every operational backend path and schema.

Observed negative or boundary output

The generated contract advertises 400/403 responses for create-download URL rejection, but live invalid URLs were accepted with 202 and failed later.

Result

PARTIAL - Documentation is available, but parts of the create-download contract do not match runtime timing.

Manual testing notes

- Import /openapi.json into an API client for discovery.
- Treat response examples as contract targets and reconcile them with asynchronous behavior.

3. Media Format Discovery

POST /api/downloads/media-info

Feature purpose

Inspects public media with yt-dlp and returns title, thumbnail, duration, and selectable audio/video formats.

How the frontend uses it

The download form calls this after client-side URL validation, then renders Video and Audio format buttons.

Request to send

```
POST /api/downloads/media-info
Content-Type: application/json

{"url": "https://youtu.be/RLzC55ai0eo"}
```

Observed positive output

```
HTTP 200 in 5,786 ms
title: "Heeriye (Official Video)..."
duration: 199.0
formats: 360p MP4 (6,872,833 bytes), MP3 320/192/128 kbps
```

Observed negative or boundary output

```
Malformed URL -> HTTP 400 INVALID_URL.
Loopback URL http://127.0.0.1:8001/api/health -> HTTP 200 with title "health" and generated MP3 options.
```

Result

FAIL - Functional behavior works, but this endpoint bypasses SSRF validation and can read private/local HTTP targets.

Manual testing notes

- This is the highest-severity finding in the audit.
- Call `validate_security` before `get_media_service().get_info()`.
- Retest redirects, IPv4/IPv6 loopback, RFC1918, link-local, cloud metadata, DNS rebinding, and alternate IP notation.

4. Create Download

POST /api/downloads

Feature purpose

Creates an in-memory record, returns a UUID immediately, and starts a background task. Direct-file jobs perform SSRF, metadata, extension, MIME, size, stream, and file-finalization checks. Media jobs invoke yt-dlp with the selected format.

How the frontend uses it

The format chooser sends url, format_id, and media_type. The Retry action sends only url.

Request to send

```
POST /api/downloads
Content-Type: application/json

{"url": "https://www.w3.org/WAI/ER/tests/xhtml/testfiles/resources/pdf/dummy.pdf"}
```

Observed positive output

```
HTTP 202 in 7.9 ms
id: 5fbe0af4-...
filename: dummy.pdf
status: queued
progress_percentage: 0.0
```

Observed negative or boundary output

```
Missing url -> 422 VALIDATION_ERROR.
Extra field -> 422 extra_forbidden.
media_type="text" -> 422 pattern error.
Empty, ftp, loopback, and HTML URLs -> 202 first, then asynchronous failed status.
```

Result

PARTIAL - Valid creation works, but basic URL and security rejection timing conflicts with the documented 400/403 contract.

Manual testing notes

- A 202 response means accepted for processing, not successfully downloaded.
- Validate format synchronously before record creation.
- If security remains asynchronous, document that clients must follow status/SSE for validation failure.

5. List, Filter, and Paginate Downloads

GET /api/downloads

Feature purpose

Returns newest-first records with optional status filtering and limit/offset pagination. Data is process memory only and disappears after restart.

How the frontend uses it

The page loads up to 50 items after a successful health check and displays DownloadCard components.

Request to send

```
GET /api/downloads?limit=50&offset=0
GET /api/downloads?status=completed
```

Observed positive output

```
HTTP 200
{"downloads":[],"total":0}
Initial test state was clean.
```

Observed negative or boundary output

```
status=unknown -> HTTP 400 INVALID_STATUS.
limit=0 -> 422. limit=101 -> 422. offset=-1 -> 422.
```

Result

PASS - Listing and declared query boundaries behaved as expected.

Manual testing notes

- Valid limit range is 1 through 100; offset starts at 0.
- Total is counted after status filtering and before pagination.
- Add persistence tests if history is expected to survive backend restart.

6. Download Status Detail

```
GET /api/downloads/{download_id}
```

Feature purpose

Returns the current state and metrics for one job: filename, bytes, percentage, speed, ETA, error, and timestamps.

How the frontend uses it

SSE fallback polling calls this every two seconds. Cards also use the data for terminal state and retry behavior.

Request to send

```
GET /api/downloads/5fbe0af4-fd2b-4492-8e2f-79599a68b0f2
```

Observed positive output

```
HTTP 200
status: completed
downloaded_bytes: 13264
total_bytes: 13264
progress_percentage: 100.0
error: null
```

Observed negative or boundary output

```
Unknown UUID -> HTTP 404 with DOWNLOAD_NOT_FOUND.
The same ID returned 404 after successful deletion.
```

Result

PASS - Existing, missing, completed, failed, and deleted state lookups were observed.

Manual testing notes

- The route accepts any string as an ID; it does not require UUID syntax.
- Error bodies for explicit HTTPException routes are wrapped in detail.error.

7. Real-Time Progress Events

```
GET /api/downloads/{download_id}/events
```

Feature purpose

Streams progress through SSE, sends a 15-second heartbeat during quiet periods, and closes after a terminal event.

How the frontend uses it

useDownloadEvents opens EventSource, merges events into card state, retries three times, then falls back to polling every two seconds.

Request to send

```
GET /api/downloads/{id}/events
Accept: text/event-stream
```

Observed positive output

```
HTTP 200
Content-Type: text/event-stream; charset=utf-8
Events observed:
1. status=validating
2. status=downloading
3. status=completed, bytes=13264, progress=100.0
```

Observed negative or boundary output

```
Unknown ID -> HTTP 404 DOWNLOAD_NOT_FOUND. Heartbeat was not observed because the small test file completed before 15 seconds.
```

Result

PASS - The live state sequence and terminal behavior were observed.

Manual testing notes

- Run a large controlled file to validate repeated progress and heartbeat timing.
- SSE is unauthenticated and exposes the original submitted URL and job metadata.

8. Retrieve Completed File

```
GET /api/downloads/{download_id}/file
```

Feature purpose

Serves a completed file from disk with application/octet-stream and a Content-Disposition attachment filename.

How the frontend uses it

Completed cards render a normal Save File link to avoid buffering large files in JavaScript.

Request to send

```
GET /api/downloads/{completed_id}/file
```

Observed positive output

```
HTTP 200
Content-Type: application/octet-stream
Content-Disposition: attachment; filename="dummy.pdf"
Body size: 13,264 bytes
```

Observed negative or boundary output

```
Missing ID -> 404 DOWNLOAD_NOT_FOUND. Failed/cancelled/non-completed record -> 409 DOWNLOAD_NOT_COMPLETED.
Missing disk file -> code path returns 404 FILE_NOT_FOUND.
```

Result

PASS - File bytes, headers, state guard, and missing-record behavior were verified.

Manual testing notes

- No Range request support is implemented for resuming large file retrieval.
- Authorization is absent; knowledge of an ID is sufficient to retrieve a completed file.

9. Cancel Active Download

POST /api/downloads/{download_id}/cancel

Feature purpose

Signals an asyncio cancellation flag or cancels the background task. Only queued, validating, or downloading records are eligible.

How the frontend uses it

Active cards expose a Cancel action and then refresh the card state.

Request to send

POST /api/downloads/{active_id}/cancel

Observed positive output

HTTP 200 for an active validating job. HTTP 409 INVALID_DOWNLOAD_STATE for an already completed job. HTTP 404 for an unknown job.

Observed negative or boundary output

Observed race: cancel returned HTTP 200 while status was validating; 200 ms later GET status returned failed with "The remote file was not found (HTTP 404)" instead of cancelled.

Result

FAIL - The endpoint acknowledges cancellation before the state is authoritative, and another failure can win the race.

Manual testing notes

- Check cancellation before and after each awaited validation/network step.
- Await task termination or update state to cancelled before returning 200.
- Add a deterministic slow-download cancellation integration test.

10. Delete Record and File

```
DELETE /api/downloads/{download_id}
```

Feature purpose

Deletes the stored file and removes the in-memory record, but only after the job is terminal.

How the frontend uses it

Completed, failed, and cancelled cards expose Remove. The UI removes the card after HTTP 204.

Request to send

```
DELETE /api/downloads/{terminal_id}
```

Observed positive output

HTTP 204 with an empty body. A later GET for the same ID returned 404. The downloaded test PDF was removed through this endpoint.

Observed negative or boundary output

```
Active job -> HTTP 409 INVALID_DOWNLOAD_STATE: "Cannot delete an active download. Cancel it first."  
Unknown ID -> HTTP 404 DOWNLOAD_NOT_FOUND.
```

Result

PASS - Terminal deletion, active-state protection, cleanup, and not-found behavior were verified.

Manual testing notes

- Deletion is permanent and has no restore/audit trail.
- The disk path is guarded so deletion cannot escape the configured download directory.

End-to-End Live Workflow Evidence

The following sequence was executed by the Postman/Newman collection against the running service, not simulated.

Step	Action	Observed result
1	POST a public W3C PDF	202 queued; UUID returned in 7.9 ms
2	Open SSE stream	200 text/event-stream
3	Observe state events	validating -> downloading -> completed
4	GET status	200; 13,264 of 13,264 bytes; 100%
5	GET file	200; application/octet-stream; attachment dummy.pdf; 13,264 bytes
6	Cancel completed job	409 INVALID_DOWNLOAD_STATE
7	DELETE completed job	204 empty response
8	GET deleted job	404 DOWNLOAD_NOT_FOUND

Observed lifecycle

```
POST /api/downloads
  | 202 queued
  v
SSE validating -> downloading -> completed
  |
  +-> GET /file = 13,264 bytes
  |
  +-> DELETE = 204
  |
  +-> subsequent GET = 404
```

Frontend integration

Requests routed through the frontend origin also worked: GET <http://127.0.0.1:3000/api/health?XTransformPort=8001> returned 200 healthy, and /api/downloads with the same routing parameter returned 200. The standalone frontend sample route /api returned {message: Hello, world!}.

Defect and Risk Register

ID	Severity	Finding	Evidence	Recommended fix
API-SEC-001	High	media-info bypasses SSRF protection	Loopback health URL returned HTTP 200 media metadata	Run validate_security before yt-dlp; validate redirects and resolved IPs
API-LIFE-002	High	Cancellation can end as failed	Cancel returned 200; later status was failed	Make cancellation state authoritative and await task outcome
API-CON-003	Medium	Invalid URLs accepted with 202	Empty, ftp, loopback, and HTML inputs created queued records	Validate format synchronously or correct the API contract
API-AUTH-004	High if exposed	No authentication or ownership authorization	Every route is public to any reachable client	Add identity, per-job ownership, and authorization checks
API-ERR-005	Low	Error envelope is inconsistent	Some errors use error; route HTTPExceptions use detail.error	Normalize through one exception model
QA-COV-006	Medium	No endpoint-level automated tests	52 tests pass, but test file covers helpers/enums only	Add TestClient/integration tests for every route and state
DEV-LINT-007	Low	Lint command is broken	ESLint 9 reports missing eslint.config.js/mjs/cjs	Add flat ESLint config or pin compatible tooling
DATA-008	Medium	History is volatile	Repository stores records only in process memory	Use persistent storage if restart durability is required

Detailed Defect: API-SEC-001

Title: Media inspection allows server-side requests to private/local targets

Field	Detail
Endpoint	POST /api/downloads/media-info
Precondition	Backend running on 127.0.0.1:8001
Request	{"url":"http://127.0.0.1:8001/api/health"}
Expected	403 UNSAFE_URL or equivalent block before any network access
Actual	200; title=health; audio format options generated
Root cause	media_info calls validate_format only, then calls yt-dlp. It does not call validate_security.
Impact	A reachable caller may probe internal services, loopback endpoints, or metadata targets through the backend.

Retest set

- 127.0.0.1, ::1, decimal/hex/octal IP forms, and localhost aliases.
- 10/8, 172.16/12, 192.168/16, 169.254/16, IPv6 ULA/link-local, and cloud metadata hostnames.
- Public URL redirecting to private IP and DNS answer changes between validation and fetch.

Detailed Defect: API-LIFE-002

Title: Cancellation acknowledgement does not guarantee cancelled terminal state

Field	Detail
Endpoint	POST /api/downloads/{id}/cancel
Setup	Create https://example.com/file.pdf and cancel immediately
Expected	200 response followed by terminal status cancelled
Actual	200 response showed validating; 200 ms later status was failed with remote 404
Root cause	The cancellation flag is checked after metadata validation. A validation exception can occur first and overwrite the intended outcome.
Impact	UI reports cancellation success while job history reports failure; metrics and user expectations become unreliable.

Recommended state rule

```
If cancellation has been accepted:  
  cancellation must win over later network/validation errors  
  final state must become cancelled  
  cancel response should return the authoritative updated record
```

Manual Test Matrix by Category

Category	Executed examples	Result
Positive	Health, root, docs, list, media info, create, status, SSE, file, delete	Passed except findings noted
Negative	Missing IDs, missing body field, extra field, bad media type, bad status	Expected 400/404/422 observed
Boundary	limit 0 and 101; offset -1	422 observed; valid range enforced
State	Cancel completed, file before completion, delete active	409 guards observed
Security	Loopback direct download and loopback media inspection	Direct path blocked later; media-info vulnerable
Data	Create, list/status, delete, confirm disappearance	In-memory lifecycle observed
Integration	Frontend proxy path to backend	200 observed for health and list
Performance smoke	Measured endpoint elapsed times	No load/benchmark claim made

Postman/Newman execution

```
newman run URL-Downloader-Live-Audit.postman_collection.json
requests: 36 executed, 0 transport failures
assertions: 48 executed, 46 passed, 2 failed
duration: 9.2s
average response: 126ms
```

The two assertion failures are the confirmed SSRF and cancellation defects described in this report. All remaining feature, validation, boundary, state, and cleanup assertions passed.

Python unit test execution

```
pytest -q
..... [100%]
52 passed, 1 warning in 0.39s
```

The passing suite covers scheme validation, SSRF helpers, filename sanitization/extraction, path safety, formatting helpers, status enums, and unique filepath generation. Despite its header, it does not currently send requests to the FastAPI endpoints.

Build and lint

- Next.js production build generated BUILD_ID sla9ULFSDCpQi5RfxX_KH.
- npm run lint failed before analysis because ESLint 9 could not find a flat eslint.config file.
- The pytest warning requests an explicit `asyncio_default_fixture_loop_scope` setting for future compatibility.

Feature-by-Feature Manual Execution Guide

Use these steps in Postman or another API client. Start the FastAPI backend on port 8001. Save the ID returned by create-download into a variable named downloadId.

Order	Request	What to verify
1	GET {{baseUrl}}/api/health	200, JSON, status=healthy
2	POST {{baseUrl}}/api/downloads/media-info	200, title, formats array; reject unsafe URLs after fix
3	POST {{baseUrl}}/api/downloads	202, UUID, queued state, timestamps
4	GET {{baseUrl}}/api/downloads/{{downloadId}}/events	text/event-stream and ordered state changes
5	GET {{baseUrl}}/api/downloads/{{downloadId}}	bytes/progress increase and final state
6	GET {{baseUrl}}/api/downloads	New job appears; filters and pagination work
7	GET {{baseUrl}}/api/downloads/{{downloadId}}/file	Only completed jobs return attachment bytes
8	POST {{baseUrl}}/api/downloads/{{downloadId}}/cancel	Only active jobs; final state must be cancelled
9	DELETE {{baseUrl}}/api/downloads/{{downloadId}}	Only terminal jobs; 204 then 404 on GET

Evidence to capture

- Request method, full URL, headers, body, send time, and environment.
- HTTP status, response headers, exact JSON body, and elapsed time.
- Download ID and state transitions; do not reuse stale IDs without noting it.
- For files: byte count, filename, Content-Type, Content-Disposition, and checksum if integrity matters.
- For defects: expected result, actual result, reproducibility, screenshots/logs, severity, and cleanup performed.

Final Assessment and Release Recommendation

Core local download functionality is demonstrably working through Postman/Newman: discovery, health, media format lookup, asynchronous creation, live progress, status retrieval, file serving, state guards, and deletion all produced usable results. The implementation is suitable for continued local development and learning.

Do not expose this backend to untrusted networks in its current form. The confirmed media-info SSRF bypass and complete absence of authentication/ownership controls create material security risk. Cancellation semantics and contract consistency should also be fixed before treating the API as production-ready.

Recommended release gate

Gate	Condition
Security	Block private/local targets in media-info and add regression tests
Authorization	Require identity and enforce per-download ownership
Lifecycle	Cancellation always reaches cancelled after 200 acknowledgement
Contract	OpenAPI, error envelope, and actual rejection timing agree
Automation	Endpoint tests cover every route, state transition, and high-risk negative case
Tooling	Lint executes successfully; build and pytest remain green

Files inspected

Primary evidence came from `src/lib/api.ts`, `src/lib/validation.ts`, `src/hooks/useDownloadEvents.ts`, `src/components/DownloadForm.tsx`, `src/components/DownloadCard.tsx`, `app/main.py`, `app/api/downloads.py`, `app/api/health.py`, `app/schemas/download.py`, `app/services/download_manager.py`, `app/services/url_validator.py`, `app/services/media_service.py`, `app/core/security.py`, `app/storage/download_repository.py`, and `tests/test_download_api.py`.

End of report